PaymentProcessor.java

Line-by-Line Code Explanation

```
public class PaymentProcessor {
```
''' The main Execution Engine. Notice it does NOT "extend" anything. It is an independent manager.

```
private List<Payment> queue = new ArrayList<>();
```
''' Polymorphic Storage: A list that holds pending transactions. Because it expects a generic "Payment", it can hold CreditCards, PayPals, and BankTransfers all at the same time.

```
private List<Payment> history = new ArrayList<>();
```
''' A second list to store transactions after they have finished processing.

```
public void addPayment(Payment payment) {
    queue.add(payment);
}
```
''' A method that accepts any child class of Payment and adds it to the waiting queue.

```
public List<ProcessResult> processAll() {
```
''' The core algorithm that runs all pending transactions.

```
List<ProcessResult> results = new ArrayList<>();
```
''' Creates an empty list to hold the final outcomes of each transaction.

```
for (Payment payment : queue) {
```
''' A loop that iterates over every single item currently waiting in the queue.

```
BiConsumer<String, String> log = (text, kind) -> lines.add(...);
```
''' Creates a lambda function (a mini-method) that we pass into the payment to capture its text output (like "Authorized").

```
boolean success = payment.process(log);
```
''' DYNAMIC DISPATCH (The Magic Line): Java looks at the generic "payment" variable, automatically detects if it's a CreditCard or PayPal in memory, and triggers that specific class's custom process() code.

```
history.add(payment);
results.add(...);
```
''' Moves the finished payment out of the queue and into the history log, recording whether it succeeded or failed.

```
queue.clear();
return results;
```
''' Empties the pending queue so it's ready for the next batch, and returns the results to the user interface.

```
public double totalSuccessful() {
```
''' A method that calculates the total amount of money successfully processed.

```
return history.stream()
    .filter(p -> "SUCCESS".equals(p.getStatus()))
    .mapToDouble(Payment::getAmount)
    .sum();
```
''' Uses Java Streams to loop through the history list, filter out the FAILED transactions, extract the dollar amounts from the successful ones, and mathematically sum them together in one fluent chain.